

Vue 3中defineExpose的使用、注意事项及与Vue 2的对比

一、defineExpose 核心作用

defineExpose 是Vue 3 `<script setup>` 语法中用于**显式暴露子组件的属性、方法**给父组件的编译宏，默认情况下 `<script setup>` 中的内容是封闭的，父组件无法直接访问子组件内部的变量/方法，必须通过 `defineExpose` 主动暴露。

二、基础使用步骤

1. 子组件（Child.vue）：定义并暴露属性/方法

Code block

```
1  <template>
2    <div>
3      <h3>子组件: {{ count }}</h3>
4      <button @click="increment">子组件内部自增</button>
5    </div>
6  </template>
7
8  <script setup>
9    import { ref, reactive } from 'vue'
10
11    // 1. 定义需要暴露的属性
12    const count = ref(0)
13    const userInfo = reactive({
14      name: '张三',
15      age: 20
16    })
17
18    // 2. 定义需要暴露的方法
19    const increment = () => {
20      count.value++
21    }
22
23    const resetCount = (num) => {
24      count.value = num
25    }
26
27    // 3. 显式暴露（核心）：父组件可访问暴露的内容
```

```
28  defineExpose({
29    count,          // 暴露响应式变量
30    userInfo,       // 暴露响应式对象
31    increment,      // 暴露方法
32    resetCount      // 暴露带参数的方法
33  })
34  </script>
```

2. 父组件（Parent.vue）：访问子组件暴露的内容

父组件需要通过 `ref` 绑定子组件实例，才能访问暴露的属性/方法：

Code block

```
1  <template>
2    <div>
3      <h2>父组件</h2>
4      <!-- 绑定 ref 标识子组件 -->
5      <Child ref="childRef" />
6
7      <!-- 父组件操作子组件暴露的内容 -->
8      <button @click="handleGetChildData">获取子组件数据</button>
9      <button @click="handleCallChildMethod">调用子组件方法</button>
10     <button @click="handleResetCount">重置子组件count</button>
11   </div>
12 </template>
13
14 <script setup>
15 import { ref } from 'vue'
16 import Child from './Child.vue'
17
18 // 定义 ref 指向子组件实例
19 const childRef = ref(null)
20
21 // 1. 获取子组件暴露的属性
22 const handleGetChildData = () => {
23   if (childRef.value) {
24     console.log('子组件count: ', childRef.value.count)
25     console.log('子组件userInfo: ', childRef.value.userInfo)
26   }
27 }
28
29 // 2. 调用子组件暴露的无参方法
30 const handleCallChildMethod = () => {
31   childRef.value?.increment()
32 }
33
```

```
34 // 3. 调用子组件暴露的带参方法
35 const handleResetCount = () => {
36   childRef.value?.resetCount(10)
37 }
38 </script>
```

三、关键注意事项

1. 响应式保持

暴露的 `ref` / `reactive` 数据在父组件中仍保持响应式，父组件修改后子组件会同步更新（反之亦然）：

Code block

```
1 // 父组件中修改子组件的响应式数据
2 const modifyChildData = () => {
3   childRef.value.count = 99 // 子组件count会同步更新
4   childRef.value.userInfo.name = '李四' // 子组件userInfo同步更新
5 }
```

2. 异步获取子组件实例

- 如果子组件有条件渲染（如 `v-if`），需确保组件挂载后再访问：

Code block

```
1 import { onMounted, nextTick } from 'vue'
2
3 // 方式1: 挂载后访问
4 onMounted(() => {
5   console.log('挂载后: ', childRef.value?.count)
6 })
7
8 // 方式2: 修改DOM后异步访问（如v-if切换后）
9 const showChild = ref(false)
10 const toggleChild = async () => {
11   showChild.value = true
12   await nextTick() // 等待DOM更新
13   console.log('子组件挂载后: ', childRef.value?.count)
14 }
```

3. 避免过度暴露

`defineExpose` 应仅暴露必要的内容，避免暴露内部私有逻辑，否则会增加组件耦合度，违背组件封装原则。

4. 类型标注（TS 场景）

如果使用 TypeScript，可通过 `defineExpose` 结合接口标注暴露内容的类型：

Code block

```
1  <script setup lang="ts">
2  import { ref, reactive } from 'vue'
3
4  interface Exposed {
5    count: number
6    userInfo: { name: string; age: number }
7    increment: () => void
8    resetCount: (num: number) => void
9  }
10
11  const count = ref(0)
12  const userInfo = reactive({ name: '张三', age: 20 })
13  const increment = () => count.value++
14  const resetCount = (num: number) => count.value = num
15
16  // 类型标注
17  defineExpose<Exposed>({
18    count,
19    userInfo,
20    increment,
21    resetCount
22  })
23  </script>
```

四、对比 Vue 2 的区别

Vue 2 选项式 API	Vue 3 <code><script setup></code> + <code>defineExpose</code>
通过 <code>this.\$refs</code> 访问	通过 <code>ref</code> 绑定 + <code>defineExpose</code> 暴露
无需显式暴露（默认开放）	必须显式暴露（默认封闭）
暴露内容无类型提示	可结合 TS 做类型标注

五、总结

`defineExpose` 是 Vue 3 中实现父子组件实例通信的重要方式，核心是**显式暴露子组件内容**，配合父组件的 `ref` 绑定即可访问。使用时需注意响应式保持、异步访问时机，同时遵循组件封装原则，仅暴露必要内容。

（注：文档部分内容可能由 AI 生成）